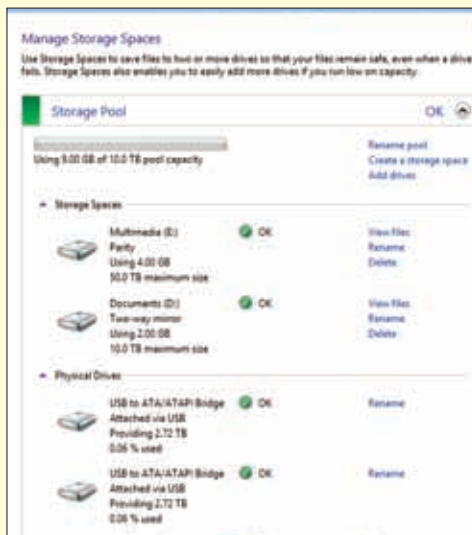


rebuild your mirrored data on it to ensure that your desired level of resiliency is maintained.

Of course, this also means that for any data stored on a mirrored partition, double the space would be used up – or even triple. This might be OK for really important, or really small data; however, for large data sets such as videos this can be really inefficient and expensive. For those, Windows supports another form of resiliency, parity. To know more about what parity means, check out the box on the side. Parity has lower storage space requirements than mirroring, which blindly maintains multiple copies; however, it is more CPU intensive, and performs a lot more random I/O.

Working in Sync

If you're excited about these new features, know that many of these have been available since as far back as Windows 2000! So why bother getting excited?



A STORAGE POOL OF MULTIPLE EXTERNAL HARD DISKS

As we said before, it's not about these new features, but how everything works together that is more interesting. For one, all of this is now user-facing, with a graphical, easy to use interface in "Control Panel". Secondly, most of this is transparent to the user once set up.

All the user has to do is connect a new drive, and add it to the storage pool. You can even mix up USB drives with internal ones, so you can continue expanding even when your internal capacity for adding drives is reduced.

With the combination of these features it is also easier to continue expanding your storage capacity even when you run out of internal and external ports. If you have enabled mirroring or parity on all your storage volumes, you can just remove your smallest hard drive and replace it with a larger one. Since the data is stored in a redundant manner, there will be no data loss,

*pointers

>>Nuggets of cool code at work.



*A Look at Tizen

>>The Linux Foundation has released the Tizen SDK, a smartphone and tablet OS. Tizen would support HTML5 apps. Duration > 09:16

<http://dvwx.in/feb-12-p1>



*Upcoming Firefox Developer Tools

>>Tilt lets you visualise web pages. It creates a 3D representation of the document, to display, and analyse the DOM. Duration > 02:06

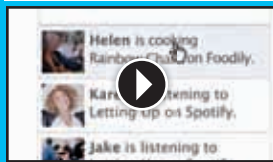
<http://dvwx.in/feb-12-p2>



*Early look at KDE 4.8

>>The KDE desktop is best known for its Plasma Desktop, the default working environment available on several Linux distros. Duration > 05:47

<http://dvwx.in/feb-12-p3>



*New class of social apps on Facebook

>>Facebook recently unveiled the Timeline, and now apps you could use with the new UI. Express yourself through all the activities you do – the music you love, the recipes you're trying out, the morning sprint, and more. Duration > 01:18

<http://dvwx.in/feb-12-p4>

Parity

>>Parity is best explained, beginning with the basic parity bit. How can you be sure that the data being transferred is error free? Comparing it with the original data is not an option, so the indications have to be within the data itself. The parity bit is one such mechanism. A parity bit – a single zero or one – is added to any transmission in order to help detect errors. It does this by ensuring that the number of 1s in any sequence of bits is either always even, or always odd.

For example, on some transmission channels which maintain even parity, a 1 or 0 could be added after every 7 bits to ensure that the number of 1s in any consecutive 8-bit sequence is even.

So, for a sequence of 001011, a parity bit of 1 will be inserted, making the sequence 1001011. In a sequence 1011010, a 0 will be added, making the sequence 01011010. In case of an error it is likely that you will get a sequence of 8 bits that has an odd number of ones, you'll know there's been an error.

We will not know what that error is, nor can it be corrected; but it can be detected. Now let's take another example, a block of 7x7 bits of data:

0	0	1	0	1	1	1	0
1	0	1	1	1	1	0	0
0	1	1	0	0	0	1	1
1	1	1	0	0	1	0	0
0	1	0	1	0	0	0	0
0	0	0	1	1	0	0	0
1	0	1	1	0	0	1	0
1	1	1	0	1	1	1	

This table has one parity bit added for each row and one for each column. Now if only one bit in this table is disturbed, the parities won't match, and by intersecting the row and column with the wrong parities, it is possible to isolate the incorrect bit and recover from the error.

This is an extremely simple and flawed example but it serves to illustrate a point; that using 14 bits of data we can detect errors and correct as many as 49 bits of data. A similar concept is used in the Windows file systems and other parity systems to recover from errors in large amounts of data using much lesser than an entire copy.

and Windows will automatically detect the new storage added to the pool and start copying data back to it to ensure that the level of redundancy you required is maintained. >